

Reto técnico — Sistema de reservas de salas

Duración: 2 días (48 horas desde la entrega de este documento) **Stack obligatorio:** Next.js (App Router) · TypeScript · Tailwind CSS · Supabase (Postgres + Auth + RLS) · GitHub + GitHub Actions

0. Lee esto primero

Podés usar IA. No está prohibido y no vamos a revisar si la usaste.

Lo que sí vamos a hacer es una **defensa en vivo de 30 minutos** al final (ver sección 9), donde vas a tener que explicar decisiones concretas de tu código y **hacer cambios en vivo** sobre tu propio proyecto. Si no entendés lo que entregaste, se va a notar ahí, no antes.

Por eso el consejo es simple: **entregá menos, pero enténdelo todo**. Un proyecto con 3 funcionalidades sólidas que sabés defender vale más que uno con 10 que no podés explicar.

1. Contexto

Un coworking en San José necesita una app para que sus miembros reserven salas de reuniones por bloques de tiempo. Hoy lo hacen por WhatsApp y se les cruzan las reservas constantemente.

Tu trabajo es construir la aplicación.

2. Roles y funcionalidades

2.1 Autenticación

- Registro con email y contraseña usando Supabase Auth.
- Inicio y cierre de sesión.
- Las rutas privadas no deben ser accesibles sin sesión. Tampoco por URL directa.
- Cada usuario tiene un rol: `miembro` o `admin`.

- El rol **no** se elige en el registro. Todo usuario nuevo nace como `miembro` . El admin se asigna manualmente en la base de datos.

2.2 Miembro

#	Funcionalidad
M1	Ver la lista de salas activas con su capacidad
M2	Ver la disponibilidad de una sala en un día específico (vista de calendario o de bloques horarios)
M3	Crear una reserva seleccionando sala, fecha, hora de inicio y duración
M4	Ver sus propias reservas: próximas y pasadas, separadas
M5	Cancelar una reserva propia (respetando la regla RN-07)
M6	Ver cuántas reservas le quedan disponibles en la semana actual

2.3 Admin

#	Funcionalidad
A1	Todo lo que puede hacer un miembro
A2	Crear una sala (nombre, capacidad, estado)
A3	Editar una sala
A4	Desactivar una sala (no se borra: se desactiva)
A5	Ver todas las reservas de todas las salas, con filtro por sala y por fecha
A6	Cancelar la reserva de cualquier usuario, indicando un motivo

3. Reglas de negocio

Estas reglas son el corazón del reto. **Todas deben cumplirse en el servidor o en la base de datos.** Si una regla solo vive en el frontend, cuenta como no implementada.

ID	Regla

RN-01	Una sala no puede tener dos reservas activas que se solapen, ni siquiera parcialmente
RN-02	Las reservas se hacen en bloques de 30 minutos (07:00, 07:30, 08:00...)
RN-03	Duración mínima: 1 hora. Duración máxima: 3 horas
RN-04	El coworking opera de 07:00 a 21:00. Ninguna reserva puede empezar antes ni terminar después
RN-05	No se puede reservar en el pasado, ni con menos de 30 minutos de anticipación
RN-06	Un miembro puede tener máximo 3 reservas activas por semana
RN-07	Una reserva se puede cancelar hasta 2 horas antes de su inicio. Después de eso, ya no
RN-08	Una reserva cancelada sí libera el bloque horario para otros usuarios
RN-09	No se pueden crear reservas nuevas en una sala desactivada
RN-10	Toda la lógica de fechas y horas opera en zona horaria de Costa Rica (<code>America/Costa_Rica</code> , UTC-6, sin horario de verano)

Decisiones que tenés que tomar vos

Estas situaciones no tienen respuesta en el enunciado a propósito. Elegí una salida, implementala y **justificala por escrito** en `DECISIONES.md` :

- **D1** — ¿Qué es "una semana" para RN-06? ¿Lunes a domingo? ¿Los últimos 7 días? ¿Qué pasa si una reserva cae justo en el límite entre dos semanas?
- **D2** — ¿Las reservas canceladas cuentan para el límite de RN-06?
- **D3** — Un admin desactiva una sala que tiene 4 reservas futuras. ¿Qué pasa con esas reservas?
- **D4** — ¿El admin está sujeto al límite de RN-06 cuando reserva para sí mismo?
- **D5** — Un usuario tiene la pantalla abierta y el bloque que quería se ocupa mientras él llenaba el formulario. ¿Qué ve? ¿Cómo se entera?

No hay respuesta correcta única. Lo que se evalúa es que la decisión sea coherente, esté implementada y esté argumentada.

4. Casos borde obligatorios

Tu entrega debe manejar estos escenarios. Vamos a probarlos.

1. **Concurrencia** — Dos usuarios envían una reserva para la misma sala, mismo horario, en el mismo instante. Exactamente una debe tener éxito y la otra debe recibir un error claro.
2. **Bypass de la UI** — Alguien llama tu API o Supabase directamente con un token válido de miembro e intenta:
 - crear una reserva a nombre de otro usuario,
 - cancelar la reserva de otro usuario,
 - crear una sala sin ser admin,
 - saltarse el límite semanal de RN-06,
 - reservar de 22:00 a 23:00.

Los cinco intentos deben fallar del lado del servidor.

3. **Solape parcial** — Existe una reserva de 09:00 a 11:00. Intentar reservar de 10:00 a 12:00 debe fallar. De 08:00 a 09:00 debe funcionar.
4. **Datos basura** — Fecha inválida, duración negativa, `sala_id` inexistente, campos vacíos. Ninguno debe reventar la app ni insertar basura.
5. **Estados vacíos** — Usuario sin reservas, sala sin disponibilidad ese día, cero salas activas. Cada uno con su mensaje, no con una tabla en blanco.

5. Requisitos técnicos

5.1 Base de datos

- Diseño de tablas propio. Como mínimo debe existir: usuarios/perfiles, salas, reservas.
- **RLS activo en todas las tablas.** Sin excepción. Una tabla sin RLS es una tabla pública.
- Políticas separadas y explícitas por operación (`select` , `insert` , `update` , `delete`).

- Migraciones versionadas en el repo (carpeta `supabase/migrations`). Nada de "lo hice a mano en el dashboard".
- Tipos de TypeScript generados desde la base, no escritos a mano:

```
npx supabase gen types typescript --project-id <id> > types/database.ts
```

5.2 Next.js

- App Router.
- Server Components por defecto. `'use client'` solo donde haga falta y lo más abajo posible en el árbol.
- La escritura de datos va por Server Actions o Route Handlers. **Nunca** un `insert` de Supabase disparado directo desde un componente de cliente sin validación de servidor.
- Validación de entrada en el servidor con un esquema (Zod u otro). No confiar en nada que venga del cliente.
- Estados de carga y error visibles. `loading.tsx` / `<Suspense>` donde corresponda.
- Responsive real. Se va a revisar en un celular de 390px de ancho.

5.3 TypeScript

- `strict: true`.
- Cero `any`. Cero `@ts-ignore`. Cero castings forzados para callar al compilador.
- `npx tsc --noEmit` debe pasar limpio.

5.4 Seguridad

- `SUPABASE_SERVICE_ROLE_KEY` jamás en código de cliente ni en variables `NEXT_PUBLIC`.
- Ningún secreto commiteado. `.env.local` en `.gitignore`, `.env.example` sí commiteado.
- El rol del usuario se verifica en el servidor, no se lee de un estado del cliente.

6. Git y CI

- Rama `main` protegida. **Cero commits directos a `main`**.

- Rama `dev` como base de trabajo. Ramas de feature salen de `dev` (`feat/reservas` , `fix/rls-salas`).
- Todo llega a `main` mediante Pull Request. **Mínimo 4 PRs.**
- **Mínimo 15 commits** con mensajes descriptivos en formato convencional (`feat:` , `fix:` , `chore:` , `refactor:`).
- El historial debe mostrar progreso incremental. Un commit inicial de 3.000 líneas es una bandera roja, aunque el código sea bueno.
- GitHub Actions corriendo en cada PR hacia `dev` y hacia `main` :

```
- npx tsc --noEmit
- npx eslint .
- npm run build
```

Si el pipeline está en rojo, el PR no se mergea. Sin excepciones.

7. Entregables

En la raíz del repositorio:

1. **README.md** con:
 - Cómo levantar el proyecto desde cero (pasos reales, probados).
 - Diagrama o descripción del modelo de datos.
 - Qué quedó funcionando y **qué quedó fuera por tiempo** (esto suma, no resta).
2. **DECISIONES.md** — bitácora con una entrada por cada decisión D1 a D5 y por cada decisión técnica relevante. Formato por entrada:


```
## D1 — Definición de semana para el límite de reservas
Fecha/hora:
Opciones consideradas:
Decisión:
Por qué:
Qué se sacrifica con esta decisión:
```
3. **SEGURIDAD.md** — explicación en tus palabras de cómo funciona cada política de RLS que escribiste y qué ataque específico bloquea cada una.

4. `scripts/seed.ts` — script que crea datos de prueba: al menos 3 salas, 2 miembros, 1 admin y 10 reservas repartidas.
 5. `scripts/concurrencia.ts` — script que dispara 10 solicitudes simultáneas de reserva sobre el mismo bloque de la misma sala e imprime cuántas tuvieron éxito. El resultado esperado es 1. Pegá la salida del script en el README.
 6. **Despliegue en Vercel** funcionando, con el link en el README.
-

8. Alcance: qué priorizar

Si el tiempo se acaba, este es el orden:

Imprescindible (sin esto no hay entrega): Auth funcionando · RLS activo · crear reserva sin solapes · ver mis reservas · CI en verde

Importante: Cancelar reserva · límite semanal · panel de admin · vista de disponibilidad

Opcional (solo si sobra tiempo): Realtime · notificaciones · filtros avanzados · animaciones

Es mejor entregar el bloque imprescindible pulido y documentado que los tres a medias.

9. Defensa en vivo (30 minutos)

Al terminar el plazo tenés una sesión de 30 minutos. Vas a compartir pantalla con tu proyecto abierto.

Parte 1 — Explicación (15 min). Te vamos a pedir que abras archivos específicos de tu repo y expliques qué hace cada parte, por qué la resolviste así y qué otra opción descartaste.

Parte 2 — Modificación en vivo (15 min). Te vamos a pedir un cambio pequeño sobre tu propio código y lo tenés que hacer en ese momento. Por ejemplo: cambiar una regla de negocio, agregar un campo, ajustar una política de RLS o arreglar un caso borde que se nos ocurra ahí.

No se evalúa velocidad. Se evalúa si sabés dónde tocar y por qué.

10. Cómo se califica

--	--	--

Criterio	Peso	Qué miramos
Integridad de datos	25%	Cómo garantizás RN-01 y RN-06. Qué pasa bajo concurrencia
Seguridad y RLS	20%	Los 5 intentos de bypass de la sección 4.2
Defensa en vivo	20%	Entendimiento real de lo que entregaste
TypeScript y arquitectura	15%	Tipado honesto, carpetas claras, lógica fuera de la UI
Git y CI	10%	Historial incremental, PRs, pipeline verde
Frontend	10%	Estados de carga/error/vacío, responsive, usabilidad

11. Descalificación automática

- Commits directos a `main`.
- Secretos (claves, tokens) commiteados en el repo.
- Alguna tabla sin RLS.
- Un solo commit gigante sin historial.
- `README.md` o `DECISIONES.md` ausentes.

12. Preguntas

Si algo del enunciado te parece ambiguo, **tomá una decisión, documentala en `DECISIONES.md` y seguí**. Parte del reto es avanzar sin tener todas las respuestas.

Si algo está genuinamente bloqueado (acceso, permisos, cuentas), escribí y se resuelve.

Suerte.